NEURAL MODELS LAB

Learning, Depth, Activations, and Output Layers

Student	Abinaya Subramani
Campus ID	2024A7PS0512G
Framework	PyTorch 2.14.1+cpu
Model	2 inputs → 2 hidden units → 1 binary logit; three-logit extension for Task 5
Training setup	Adam, learning rate 0.03, seed 1 for binary experiments, 10,000 full-batch steps

1. Task 1 — Problem specification

The model represents a warning system with two binary sensors. The output is 1 exactly when the sensor values disagree (XOR).

Input (x■, x■)	Target y
(0, 0)	0
(0, 1)	1
(1, 0)	1
(1, 1)	0

Input space: X = {(0,0), (0,1), (1,0), (1,1)}. **Output space:** Y = {0,1}. Labels are [0,1,1,0].

Linear separability: The positive examples (0,1) and (1,0) lie on opposite corners, while negative examples (0,0) and (1,1) occupy the other corners. No single straight line separates the classes. A single affine transformation followed by sigmoid cannot represent XOR; stacking affine-only layers still gives an affine transformation. A nonlinear hidden activation is necessary.

Prediction: a single affine layer followed by sigmoid will not correctly classify all four XOR examples, regardless of training duration.

2. Task 2 — Model design and validation

Binary architecture: 2 input features → 2 hidden units → 1 output logit, with sigmoid hidden activation. BCEWithLogitsLoss combines sigmoid and binary cross-entropy in a numerically stable loss.

Training: full-batch Adam, learning rate 0.03, 10,000 steps, random initialisation, seed 1. The model returns logits; sigmoid is applied only when reporting probabilities.

Validation criteria: compare initial/final loss; check all four labels; inspect an early gradient norm and a gradient tensor; test zero-initialisation symmetry; compare sigmoid, tanh and ReLU.

Why nonlinearity? Hidden units can form nonlinear features that make XOR separable in the hidden representation. Backpropagation adjusts hidden parameters according to how they affect output loss.

Output/loss pairing: one binary logit with binary cross-entropy is suitable for yes/no classification; sigmoid converts the logit to a probability.

3. Task 3 — LLM use and code inspection

The exact original prompt was not separately supplied. The prompt below is the one recorded in the earlier draft; verify it matches the prompt actually used before submission.

```
Generate minimal PyTorch code for the four XOR examples (0,0)->0,
(0,1)->1, (1,0)->1, (1,1)->0. Use a 2-input, 2-hidden-unit, 1-output
network with sigmoid hidden activation, one output logit trained using
BCEWithLogitsLoss, random weight initialisation, a fixed random seed,
and full-batch Adam training for 10,000 steps on CPU. Print initial/final
loss, all four probabilities, thresholded labels, and the first-layer
weight gradient norm. Also test all-zero initialisation and compare
sigmoid, tanh, and ReLU while changing only the hidden activation.
Finally implement a 3-logit output using CrossEntropyLoss for classes
(0,0)->0, (0,1)->1, (1,0)->1, (1,1)->2. Do not change the task or
architecture beyond the stated output extension.
```

Code inspection: the forward pass computes logits; BCEWithLogitsLoss forms the scalar loss; loss.backward() computes parameter gradients using reverse-mode automatic differentiation; opt.step() updates parameters. The code uses fixed seeds and reports numerical checks. Human verification is necessary to ensure the labels, architecture, loss and interpretation match the task.

4. Task 4A–B — Binary XOR and gradient evidence

The binary model learned the XOR mapping successfully in the supplied run.

Measurement	Observed result
PyTorch	2.14.1+cpu
Initial loss	0.70276976
Final loss	0.00001149
Probabilities	[0.00001558, 0.99998891, 0.99998903, 0.00000905]
Predicted labels	[0, 1, 1, 0]
Target labels	[0, 1, 1, 0]
All correct	Yes
Early first-layer gradient L2 norm	0.0043040058

Interpretation: loss decreased from 0.70276976 to approximately 1.149 × 10■■ and all labels match the targets. The nonzero early gradient norm indicates a learning signal. Very small gradients after training are consistent with convergence.

Gradient meaning: parameter.grad contains ∂L/∂W■[1]■. Since loss is averaged over the four examples, the batch gradient is the average of example-wise gradients.

First-layer weight gradient after training:

```
[[-1.0900e-08, -1.4389e-08],
 [-1.4177e-07, -1.6387e-07]]
```

5. Task 4C — Zero-initialisation symmetry

Measurement	Observed result
Initial loss	0.69314718
Final loss	0.69314718
Hidden weight row 1	[0.0, 0.0]
Hidden weight row 2	[0.0, 0.0]
Rows remain identical	Yes
Probabilities	[0.5, 0.5, 0.5, 0.5]
Predictions	[1, 1, 1, 1]

Identical hidden units compute the same activations and receive identical gradients, so their parameters remain identical instead of developing distinct features. Loss remains 0.69314718 and the model does not learn XOR. Random initialisation breaks this symmetry.

6. Task 4D — Hidden activation comparison

All runs used seed 1, Adam with learning rate 0.03, and 10,000 steps; only the hidden activation changed.

Activation	Final loss	4/4 correct?	Early gradient norm	Probabilities (input order)
Sigmoid	1.149233 × 10■■	Yes	0.0043040058	[1.6e-05, 0.999989, 0.999989, 9e-06]
Tanh	0.34657902	No	0.018582033	[2e-06, 0.501993, 0.999998, 0.501994]
ReLU	0.47738576	No	0.019310528	[0.333328, 1.0, 0.333328, 0.333328]

Engineering observation: sigmoid classified all four examples correctly for this seed and configuration; tanh and ReLU did not. This does not establish that sigmoid is universally best.

Scientific explanation: sigmoid saturation can shrink derivatives; ReLU has zero derivative for negative pre-activations. A larger gradient norm alone does not guarantee a correct classifier; inspect activations, pre-activations, loss and predictions together.

7. Task 5 — Three-class extension

Class 0 represents (0,0), class 1 represents (0,1) and (1,0), and class 2 represents (1,1). The model uses two hidden units and three output logits trained with CrossEntropyLoss.

Input	Target	Softmax probabilities [0,1,2]	Predicted	Sum
(0,0)	0	[0.999995, 0.000005, 0.000000]	0	1.00000005
(0,1)	1	[0.000002, 0.999996, 0.000002]	1	0.99999995
(1,0)	1	[0.000002, 0.999996, 0.000002]	1	0.99999991
(1,1)	2	[0.000000, 0.000006, 0.999994]	2	0.99999999

Architecture: 2 inputs → 2 hidden units → 3 logits. Final weight matrix shape: (3, 2). Initial loss: 1.06920040; final loss: 0.00000504. All four predicted classes match their targets.

Softmax: p■ = exp(z■)/Σ■ exp(z■), so probabilities sum to one up to floating-point rounding. For softmax with cross-entropy, the gradient with respect to the logits is p – y, where y is the one-hot target.

8. Conclusion

The sigmoid-hidden binary model learned XOR with low loss and all four predictions correct. The zero-initialisation experiment preserved hidden-unit symmetry and failed to learn. In the activation comparison, only sigmoid reached 4/4 correct under the tested settings; this is an observation for this run, not a universal ranking of activations. The three-class extension correctly predicted all four classes and returned softmax probability vectors summing to approximately one.

The numerical results above are transcribed from the output supplied for this report. Re-run the accompanying Python file to reproduce the experiments in your own environment. Small numerical differences may occur across software versions or platforms.

Before submission: verify that the LLM prompt in Section 3 matches the prompt actually used. It was reconstructed from the earlier draft because the exact original prompt was not supplied.